

数据库系统原理

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第3章 关系数据库标准语言SQL



厚德 笃学 崇实 尚新

第3章 关系数据库标准语言SQL

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 数据查询

3.5 数据更新

3.6 视图



3.4 查 询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 **SELECT** 语句的一般格式



3.4 数据查询

■ 语句格式

SQL提供了结构形式一致但功能多样化的检索语句Select

投影

$\Pi_{A1, \dots, An}$

笛卡尔
积($\times$)

选择(σ)

SELECT [ALL|DISTINCT] <目标列表表达式> [, <目标列表表达式>] ...
FROM <表名或视图名> [, <表名或视图名>] ...
[WHERE <条件表达式>]
[GROUP BY <列名1> [HAVING <条件表达式>]]
[ORDER BY <列名2> [ASC|DESC]];

相当于 $\Pi_{\text{列名}, \dots, \text{列名}} (\sigma_{\text{检索条件}} (\text{表名1} \times \text{表名2} \times \dots))$

<目标列表表达式> 可以为: 算术表达式、子查询、聚合函数、列别名

基本查询
结构

核心映射关系: 关系代数的选择 (σ) 对应 SQL 的 WHERE, 投影 (π) 对应 SELECT 指定列, 等值连接 ($\bowtie$) 对应 JOIN ... ON, 集合操作 ($U/\cap/-$) 对应 UNION/INTERSECT/EXCEPT



3.4 数据查询

■ 语句格式

- SELECT子句：指定要显示的属性列
- FROM子句：指定查询对象（基本表或视图）
- WHERE子句：指定查询条件

GROUPBY子句：对查询结果按指定列的值分组，该属性列值相等的元组为一个组。通常会在每组中作用聚集函数。

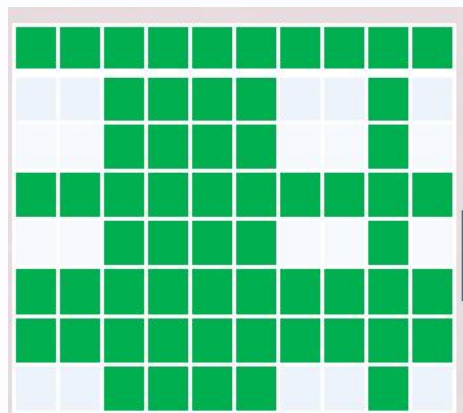
HAVING短语：只有满足指定条件的组才予以输出

ORDER BY子句：对查询结果表按指定列值的升序或降序排序

- 语义：从表名所给出的表中，查询出满足检索条件的元组，并按给定的列名及顺序进行投影显示。
- Select语句中的**select ... , from... , where...**等被称为子句，在以上基本形式基础上会增加许多构成要素，也会增加许多新的子句，满足不同的需求。



3.4 数据查询

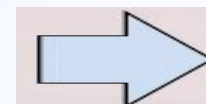


SELECT-FROM-WHERE



201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

GROUP BY-HAVING
(常伴有聚集函数)



男	2
女	2

ORDER BY



3.4.1 单表查询

查询仅涉及一个表，是一种最简单的查询操作

- 选择表中的若干列
- 选择表中的若干元组
- 对查询结果排序
- 使用集函数
- 对查询结果分组



3.4.1 单表查询

■ 查询指定列

[例1] 查询全体学生的学号与姓名。

SELECT Sno, Sname FROM Student; $\Pi_{Sno, Sname}(Student)$

Sno	Sname
202215122	刘晨
202215125	张立
202215121	李勇
202215123	王敏

[例2] 查询全体学生的姓名、学号、所在系。

SELECT Sname, Sno, Sdept FROM Student;

Sname	Sno	Sdept
李勇	202215121	CS
刘晨	202215122	CS
王敏	202215123	MA
张立	202215125	IS

[例3] 查询全体学生的详细记录。

■ 选出所有属性列

- 在**SELECT**关键字后面列出所有列名
- 将<目标列表表达式>指定为 *

SELECT Sno, Sname, Ssex, Sage, Sdept FROM Student;

或

SELECT * FROM Student;

Sno	Sname	Ssex	Sage	Sdept
202215121	李勇	男	20	CS
202215122	刘晨	女	19	CS
202215123	王敏	女	18	MA
202215125	张立	男	19	IS



查询经过计算的值

SELECT子句的<目标列表达式>为表达式

- 算术表达式
- 字符串常量
- 函数
- 列别名
- 等

● “虚”列



查询经过计算的值（续）

[例4] 查全体学生的姓名与出生年份。

```
SELECT Sname, 2026-Sage FROM  
Student;
```

Sname	2014-Sage
李勇	1994
刘晨	1995
王敏	1996
张立	1995

$\Pi_{\text{Sname, 出生年份}}(\sigma_{\text{出生年份}=2026-\text{Sage}}(\text{Student}))$

[例5] 查询全体学生的姓名、出生年份和所有系，要求用小写字母表示所有系名。

使用列别名改变查询结果的列标题：

```
SELECT Sname NAME, 'Year of Birth:'  
BIRTH, 2014-Sage BIRTHDAY,  
ISLOWER(Sdept) DEPARTMENT  
FROM Student;
```

NAME	BIRTH	BIRTHDAY	DEPARTMENT
李勇	Year of Birth:	1994	cs
刘晨	Year of Birth:	1995	cs
王敏	Year of Birth:	1996	ma
张立	Year of Birth:	1995	is

MySQL: LOWER ()



查询经过计算的值（续）

[例6] 查询中如何取消重复的行？

SELECT ALL | DISTINCT Sno FROM SC;

默认：ALL

Sno
201215121
201215121
201215121
201215122
201215122

Sno
201215121
201215122



查询满足条件的元组

WHERE子句常用的查询条件

查 询 条 件	谓 词
比 较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空 值	IS NULL, IS NOT NULL
多重条件（逻辑运算）	AND, OR, NOT



比较大小

[例7] 查询计算机科学系全体学生的名单。

```
SELECT Sname FROM Student WHERE Sdept='CS';
```

Sname
李勇
刘晨

[例8] 查询所有年龄在20岁以下的学生姓名及其年龄。

```
SELECT Sname, Sage FROM Student WHERE Sage < 20;
```

Sname	Sage
刘晨	19
王敏	18
张立	19

[例9] 查询考试成绩有不及格的学生的学号。

```
SELECT DISTINCT Sno FROM SC WHERE Grade<60;
```

Sno



确定范围

- 谓词: **BETWEEN ... AND ...**
NOT BETWEEN ... AND ...

[例10] 查询年龄在20~23岁之间（包括20岁和23岁）的学生的姓名、系别和年龄。

SELECT Sname, Sdept, Sage FROM Student WHERE Sage BETWEEN 20 AND 23;

Sname	Sdept	Sage
李勇	CS	20

[例10] 查询年龄不在20~23岁之间的学生的姓名、系别和年龄。

SELECT Sname, Sdept, Sage FROM Student WHERE Sage NOT BETWEEN 20 AND 23;

Sname	Sdept	Sage
刘晨	CS	19
王敏	MA	18
张立	IS	19



确定集合

■ 谓词: **IN <值表>, NOT IN <值表>**

[例11] 查询信息系（IS）、数学系（MA）和计算机系（CS）学生的姓名和性别。

```
SELECT Sname, Ssex FROM Student  
WHERE Sdept IN ( 'IS', 'MA', 'CS' );
```

[例11] 查询既不是计算机科学系、数学系，也不是信息系的学生们的姓名和性别。

```
SELECT Sname, Ssex  
FROM Student  
WHERE Sdept NOT IN ( 'IS','MA','CS' );  
SELECT Sname, Ssex FROM Student  
WHERE Sdept NOT IN ( 'IS', 'MA', 'CS' );
```

Sname	Ssex
-------	------



字符串匹配

■ 字符串匹配: **[NOT] LIKE** '<匹配串>' **[ESCAPE** '<换码字符>']

✓ <匹配串>: 指定匹配模板（固定字符串或含通配符的字符串）

■ 通配符

◆ % (百分号) 代表任意长度（长度可以为0）的字符串

例: **a%b**表示以**a**开头，以**b**结尾的任意长度的字符串。如**acb**，**addgb**，**ab**等都满足该匹配串

✧ _ (下横线) 代表任意单个字符

例: **a_b**表示以**a**开头，以**b**结尾的长度为**3**的任意字符串。如**acb**，**afb**等都满足该匹配串

■ **ESCAPE** 短语: 当用户要查询的字符串本身就含有 % 或 _ 时，要使用**ESCAPE** '<换码字符>' 短语对通配符进行转义。



字符串匹配

```
INSERT INTO student(Sno,Sname,Sdept,Ssex,Sage)
VALUES('202215126','欧阳杰','AI','男',18);
INSERT INTO Course (Cno,Cname,Cpno,Credit)
VALUES ('8','DB_Design','1',2);
INSERT INTO Sc VALUES('202215126','1',NULL);
```

[例12] 查询所有姓刘学生的姓名、学号和性别。

```
SELECT Sname,Sno,Ssex FROM Student
WHERE Sname LIKE '刘%';
```

Sname	Sno	Ssex
刘晨	202215122	女

[例13] 查询姓"欧阳"且全名为三个汉字的学生的姓名。

```
SELECT Sname FROM Student
WHERE Sname LIKE '欧阳__';
```

Sname
欧阳杰

[例14] 查询DB_Design课程的课程号和学分。

```
SELECT Cno,Ccredit FROM Course
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```



字符串匹配

For MySQL

```
INSERT INTO Course (Cno,Cname,Cpno,Credit)
```

```
VALUES ('9','人工%Design','1',2);
```

```
INSERT INTO Course (Cno,Cname,Cpno,Credit)
```

```
VALUES ('10','人工智能','1',2);
```

[例] `SELECT Cno,Cname,Credit FROM Course
WHERE Cname LIKE '人工%';`

Cno	Cname	Credit
10	人工智能	2
9	人工%Design	2

[例] `SELECT Cno,Cname,Credit FROM Course
WHERE Cname LIKE '人工\\%%';`

Cno	Cname	Credit
9	人工%Design	2

但: `SELECT Cno,Cname,Credit FROM Course WHERE Cname
LIKE '人工\\%%' ESCAPE '\\';`

错误代码: 1064

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''\'



涉及空值的查询

■ 涉及空值的查询： IS NULL 或 IS NOT NULL

◆ “IS” 不能用 “=” 代替

[例15] 某些学生选修课程后没有参加考试，所以有选课记录，但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno, Cno, Grade FROM SC
WHERE Grade IS NULL;
```

Sno	Cno	Grade
202215126	1	(NULL)

[例16] 查所有有成绩的学生学号和课程号。

```
SELECT Sno, Cno ,Grade FROM SC
WHERE Grade IS NOT NULL;
```

Sno	Cno	Grade
202215121	1	92
202215121	2	85
202215121	3	88
202215122	2	90
202215122	3	80



课堂问题：两个查询结果一样吗？

■ SELECT b
FROM R
WHERE a < 10 OR a >= 10;

b
4
6
8

a	b
1	4
2	6
NULL	7
10	8

SELECT b
FROM R;

b
4
6
7
8



多重条件查询

用逻辑运算符**AND**和 **OR**来联结多个查询条件

- **AND**的优先级高于**OR**
- 可以用括号改变优先级

可用来实现多种其他谓词

- **[NOT] IN**
- **[NOT] BETWEEN ... AND ...**



多重条件查询

[例17] 查询计算机系年龄在20岁以下的学生姓名。

```
SELECT Sname FROM Student WHERE Sdept= 'CS' AND Sage<20;
```

例11可改写为:

```
SELECT Sname, Ssex FROM Student  
WHERE Sdept= 'IS ' OR Sdept= ' MA' OR Sdept= ' CS ';
```

检索条件的书写

- 与选择运算 $\sigma_{con}(R)$ 的条件con书写一样，只是其逻辑运算符用 **and** , **or**, **not** 来表示，同时也要注意运算符的优先次序及括弧的使用。书写要点是注意对自然语言检索条件的正确理解。

示例：检索教师表中所有工资少于1500元或者工资大于2000元 并且是03系的教师姓名？

```
Select Tname From Teacher  
Where Salary < 1500 or Salary > 2000 and D# = '03';
```

对吗？



使用**ORDER BY**子句

- 可以按一个或多个属性列排序
- 升序：**ASC**；降序：**DESC**；缺省值为升序

➤ 当排序列含空值时

空值的显示顺序由具体系统实现来决定

例如：按升序排，含空值的元组最后显示

按降序排，空值的元组最先显示

➤ 各个系统的实现可以不同，只要保持一致即可



对查询结果排序

[例18] 查询全体学生情况，查询结果按所在系的系号升序排列，同一系中的学生按年龄降序排列。

SELECT * FROM Student ORDER BY Sdept, Sage DESC;

Sno	Sname	Sdept	Ssex	Sage
202215126	欧阳杰	AI	男	18
202215121	李勇	CS	男	20
202215122	刘晨	CS	女	19
202215125	张立	IS	男	19
202215123	王敏	MA	女	18

[例] 查询选修了3号课程的学生的学号及其成绩，查询结果按分数降序排列。

**SELECT Sno,Grade
FROM SC
WHERE Cno= '3'
ORDER BY Grade DESC;**

Sno	Grade
202215121	88
202215122	80



◆ 基于统计的查询（聚集函数）

- 统计元组个数

COUNT(*)

- 统计一列中值的个数

COUNT([DISTINCT|ALL] <列名>)

- 计算一列值的总和（此列必须为数值型）

SUM([DISTINCT|ALL] <列名>)

- 计算一列值的平均值（此列必须为数值型）

AVG([DISTINCT|ALL] <列名>)

- 求一列中的最大值和最小值

MAX([DISTINCT|ALL] <列名>)

MIN([DISTINCT|ALL] <列名>)



使用集函数

[例19] 查询学生总人数。

```
SELECT COUNT(*) FROM Student;
```

```
count( $\pi$   $s_{no}$  (Student))
```

COUNT (*)
5

[例20] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno) FROM SC;
```

COUNT (DISTINCT Sno)
3

[例21] 计算1号课程的学生平均成绩。

```
SELECT AVG(Grade) FROM SC WHERE Cno= ' 1 ';
```

AVG (Grade)
92.0000

Sno	Cno	Grade
202215121	1	92
202215121	2	85
202215121	3	88
202215122	2	90
202215122	3	80
202215126	1	(NULL)

[例22] 查询选修1号课程的学生最高分数。

```
SELECT MAX(Grade) FROM SC WHERE Cno= ' 1 ';
```

[例23] 查询学生20225012选修课程的总学分数。

```
SELECT SUM(Ccredit) FROM SC, Course  
WHERE Sno='202215122' AND SC.Cno=Course.Cno;
```

SUM (Ccredit)
6



对查询结果分组

◆ 基于分组的统计查询（**GROUP BY**子句）

- ✓ 细化聚集函数的作用对象；
- ✓ 作用对象是查询的中间结果表；
- ✓ 按指定的一列或多列值分组，值相等的为一组。

[例24] 求各个课程号及相应的选课人数。

```
SELECT Cno, COUNT(Sno) FROM SC GROUP BY Cno;
```

Cno	COUNT (Sno)
1	2
2	2
3	2



对查询结果分组（续）

- 使用GROUP BY子句后，SELECT子句的列名列表中只能出现分组属性和聚集函数，或者说，select 后面的所有列中，没有使用聚合函数的列，必须出现在 group by 后面

```
select Sno,Cno from SC group by Sno
```

--将会出现错误

--消息 8120，级别 16，状态 1，第 1 行

```
select Sno as '学号',  
COUNT(*) as '选课门数' from SC group by Sno
```

学号	选课门数
202215121	3
202215122	2
202215126	1



使用HAVING短语筛选最终输出结果

[例25] 查询选修了3门以上课程的学生学号。

```
SELECT Sno FROM SC GROUP BY Sno HAVING COUNT(*) >2;
```



Sno
202215121

- 只有满足**HAVING**短语指定条件的组才输出
- **HAVING**短语与**WHERE**子句的区别：作用对象不同
 - ✓ **WHERE**子句作用于基表或视图，从中选择满足条件的元组。
 - ✓ **HAVING**短语作用于组，从中选择满足条件的组。



HAVING字句（续）

■ [例3.48] 查询平均成绩大于等于90分的学生学号和平均成绩

➤ 下面的语句是不对的：

```
SELECT Sno, AVG(Grade)
FROM SC
WHERE AVG(Grade)>=90
GROUP BY Sno;
```

1 queries executed, 0 success, 1 errors, 0 warnings

查询: SELECT Sno,AVG(Grade) FROM SC WHERE AVG(Grade)>=90 GROUP BY Sno LIMIT 0, 1000

错误代码: 1111

Invalid use of group function

➤ 因为WHERE子句中是不能用聚集函数作为条件表达式

➤ 正确的查询语句应该是：

```
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade)>=88;
```

Sno	AVG (Grade)
202215121	88.3333



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新

下课了。。



休息。。。

